COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 5: Data compression related algorithms

Faculty of Information Technology, Monash University

What is covered in this lecture?

This week's lecture deals with...

- Introduction to lossless data compression.
- Fixed and variable length codes.
- Prefix-free codewords for encoding/decoding
  - character streams
  - integer streams
- A simple dictionary-based compression algorithm on text.

Source material and recommended reading

- MacKay, Information theory, inference and learning algorithms (chapter 1) [Link]
- Sayood, Introduction to Data compression (chapters 3&5)

Introduction

- Human data streams generate ≈**2.5 quintillion bytes** of data each day.
- This data is of various descriptions (text, images, movies, structured-tables, etc.)
- Compression of data saves **space** and **time** (in storage/retrieval/transmission etc).

Modern applications

For general files

- zip/gzip/7zip
- bzip
- pkzip
- etc.

For multimedia

- GIF, JPEG
- MPEG, DivX,
- etc.

Others

- Fax, modem
- skype, zoom
- large databases: Google, Amazon, Twitter etc.

What is compression?

- Data compression is the practice of representing information in a compact form.

What is compression?

- Data compression is the practice of representing information in a compact form.
- Compression is achieved by identifying and exploiting repeat-patterns in data.

What is compression?

- Data compression is the practice of representing information in a compact form.
- Compression is achieved by identifying and exploiting repeat-patterns in data.

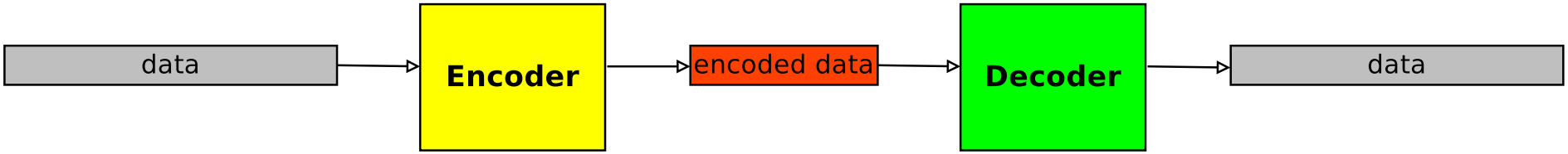
$$
\begin{aligned}
Data &= \texttt{redundant parts} + \texttt{random parts} \\
&= \texttt{compressible parts} + \texttt{uncompressible parts} \\
&= \texttt{model} + \texttt{deviations} \\
&= \texttt{signal} + \texttt{noise}
\end{aligned}
$$

Compression is a really old idea/technology

- Natural language (as old as humanity) internalizes compression:
  - Often, **commonly** articulated words
    - ★ 'yes'/'no'
    - ★ 'go'/'come',
    - ★ 'ma'/'pa')

    are shorter...
  - ...than words that are **uncommon**.
    - ★ 'piscatorial'
    - ★ 'abstemious'
    - ★ 'floccinaucinihilipilification'
- Mathematics embodies compression.
- Some old technologies (circa 1800s) for communicating textual information (where short codes are assigned to common characters, and long codes for uncommon ones):
  - Braille (tactile) code for visually impaired
  - Morse code for telegraph
  - Baudot code for teletype
  - etc.

Lossless compression – encoding and decoding



Lossless compression

Lossless data compression algorithms allow encoding the original data into a compact (encoded) form, and which in turn can be perfectly reconstructed (decoded) to get back the original data.

Fixed length codewords

Fixed length codewords

- Each symbol's codeword takes the **same** number of bits to state
- Convenient to encode and decode each symbol.
- With fixed-length codewords, we do not care about the underlying probability/frequency of each symbol's occurrence.

Fixed-length codewords example – ASCII

Decimal - Binary - Octal - Hex – ASCII Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	“	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	‘	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	|
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

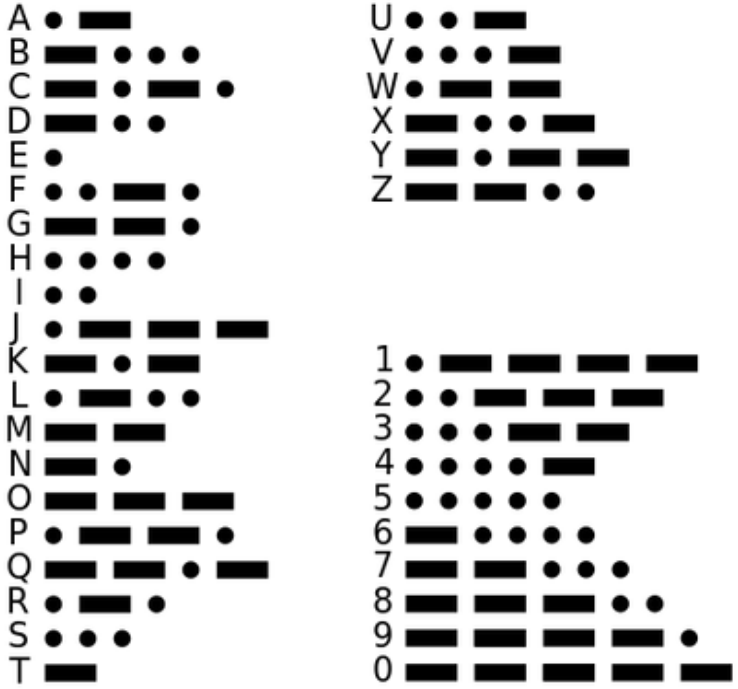
This work is licensed under the Creative Commons Attribution-ShareAlike License. To view a copy of this license, visit http://creativecommons.org/licenses/by-sa/3.0/ ASCII Conversion Chart.doc Copyright © 2008, 2012 Donald Weiman 22 March 2012

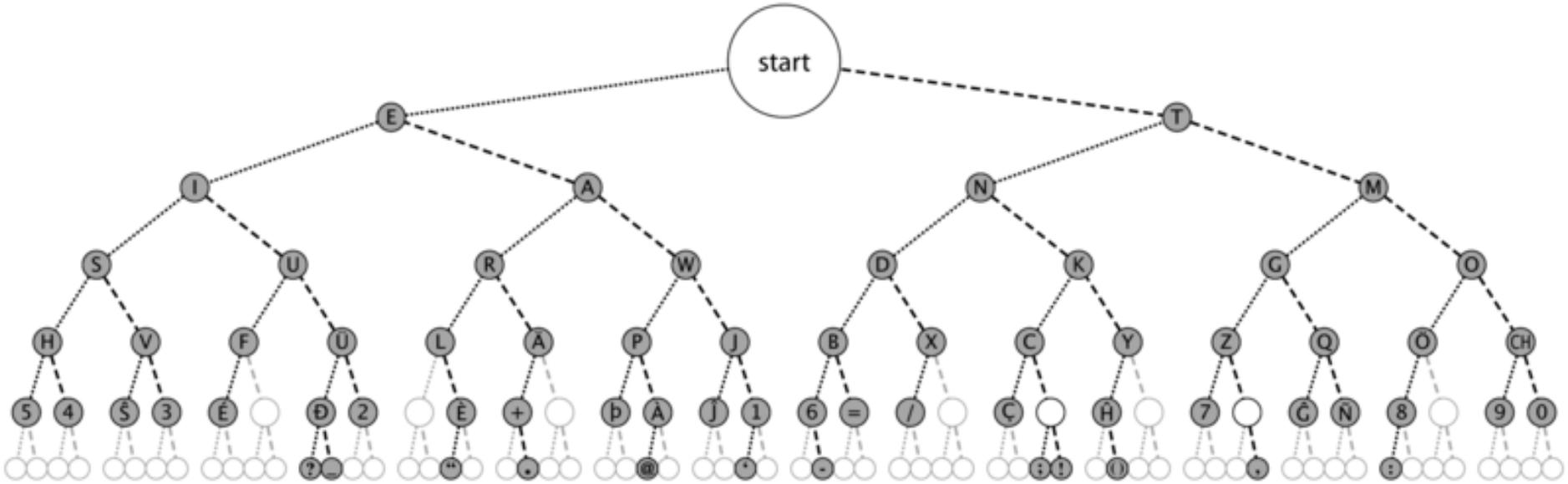
Variable length codewords

Variable length codewords

- Each symbol's codeword takes **`varying`** number of bits
- Takes more effort to encode and decode each symbol.
- Here, we have to consider the underlying probability of each symbol's occurrence, when designing the codewords.

Variable-length codewords example – Morse code

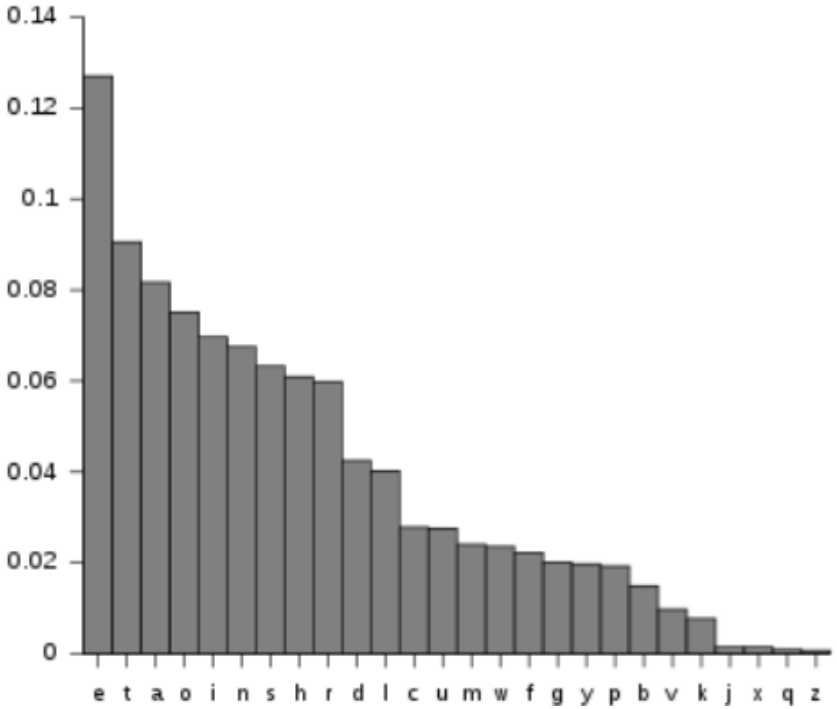




Variable length codes are central to compression

- If **frequent symbols** in some text are assigned **shorter codewords**...
- ...and **infrequent symbols** are assigned **longer codewords**...
- ...then the **total number of bits** to encode the source text/data will be **smaller**...
- ...i.e., **compressed**, compared to encoding using fixed-length codewords.
- These variable-length codewords provide the **building blocks** for data compression.

Frequency of letters in the English alphabet:



Problem with variable-length codes – decoding can become ambiguous!

Example of ambiguous variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 1, **C** = 10, **D** = 101

Encoding of **BABA** using this code is: 1010

Problem with variable-length codes – decoding can become ambiguous!

Example of ambiguous variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 1, **C** = 10, **D** = 101

Encoding of **BABA** using this code is: 1010

Decoding of this encoded message is not unique, since we don't know where each codeword starts and ends. In the above example, the message can be decoded as:

- 1 0 1 0 = **B A B A**, or

Problem with variable-length codes – decoding can become ambiguous!

Example of ambiguous variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 1, **C** = 10, **D** = 101

Encoding of **BABA** using this code is: 1010

Decoding of this encoded message is not unique, since we don't know where each codeword starts and ends. In the above example, the message can be decoded as:

- 1 0 1 0 = **B A B A**, or
- 10 10 = **C C**, or

Problem with variable-length codes – decoding can become ambiguous!

Example of ambiguous variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 1, **C** = 10, **D** = 101

Encoding of **BABA** using this code is: 1010

Decoding of this encoded message is not unique, since we don't know where each codeword starts and ends. In the above example, the message can be decoded as:

- 1 0 1 0 = **B A B A**, or
- 10 10 = **C C**, or
- 101 0 = **D A**

(spaces between codewords added above for convenience in parsing)

Prefix-free (variable-length) codes provide unique decodability

Now consider these variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 10, **C** = 110, **D** = 111

Encoding of **BABA** using this code is: 100100

Indeed this encoded message is **uniquely decodable**. Why?

Prefix-free (variable-length) codes provide unique decodability

Now consider these variable-length codewords

Assume we are using these codewords: **A** = 0, **B** = 10, **C** = 110, **D** = 111

Encoding of **BABA** using this code is: 100100

Indeed this encoded message is **uniquely decodable**. Why?

Answer: No symbol's codeword is a **prefix** of the codeword of another symbols. Such codewords are called **prefix-free** codes.

Prefix-free codes are sometimes shorthanded to '**prefix codes**', without explicitly adding '**-free**'. They are also called **instantaneous codes**

Huffman coding yields **prefix-free** codes for characters

- Huffman coding is a binary-tree based method of generating reliable **prefix-free** codewords for characters.
- It requires as input, **frequencies** of characters you are encoding.
- It yields:
  - the shortest codeword for the most frequent character.
  - the second shortest codeword for the second most frequent character.
  - ...and so on.
- Huffman coding algorithm is classified as a **greedy algorithm**.

Huffman code tree structure

- Huffman coding constructs a binary tree.
- Each character to be encoded is represented by a leaf node.
- Each internal node denotes the subset of characters (specified by the leaf nodes hanging off it).
- The path/traversal from the root node to any leaf yields a prefix-free binary 'codeword'.

Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

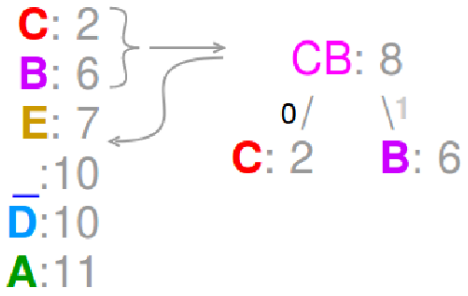
Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

2. C: 2
   B: 6
   E: 7
   _:10
   D:10
   A:11

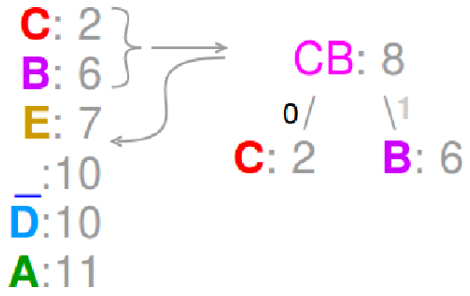
Huffmann coding by example – from [wikipedia]

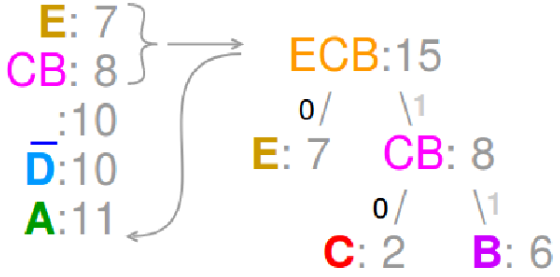
1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

2. 

Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

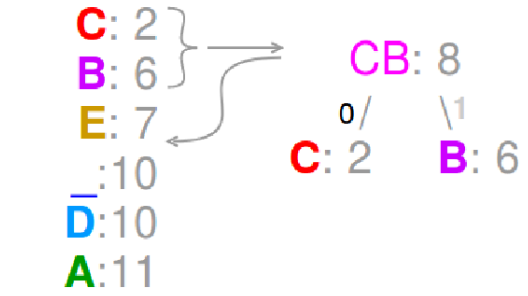
2. 

3. 

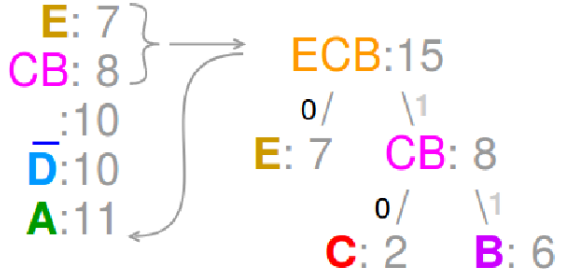
Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

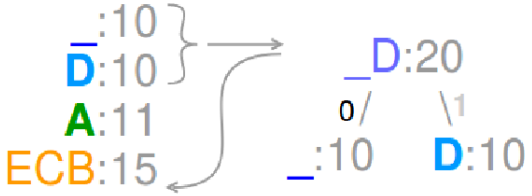
2.



3.



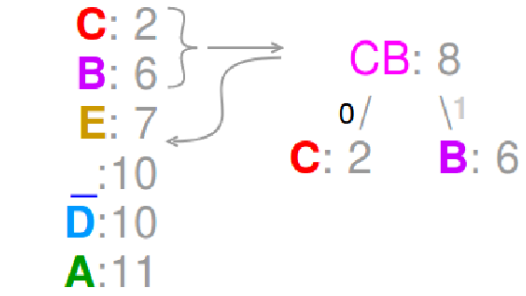
4.



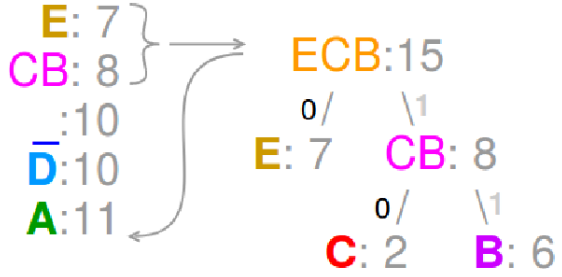
Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

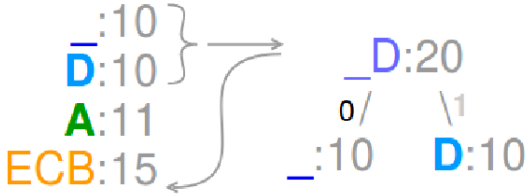
2.



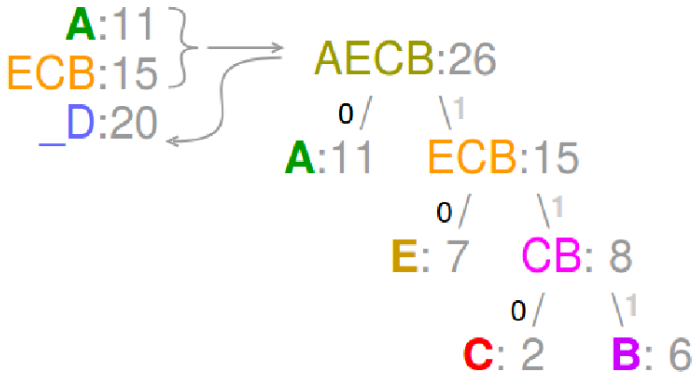
3.



4.



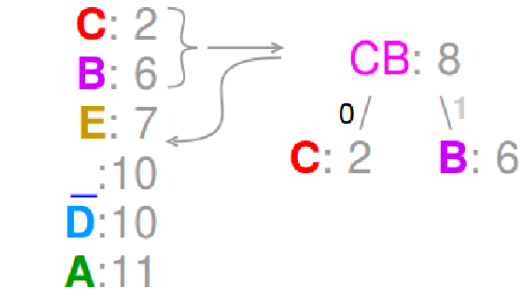
5.



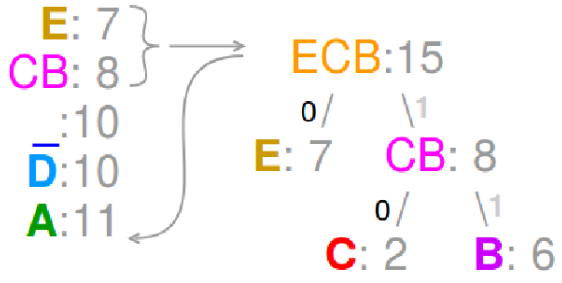
Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

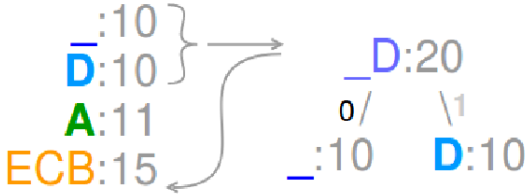
2.



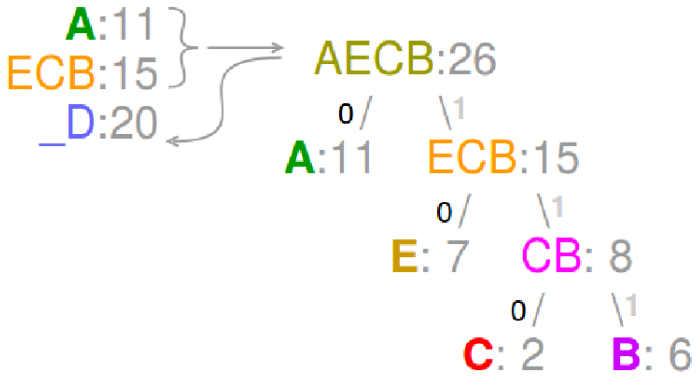
3.



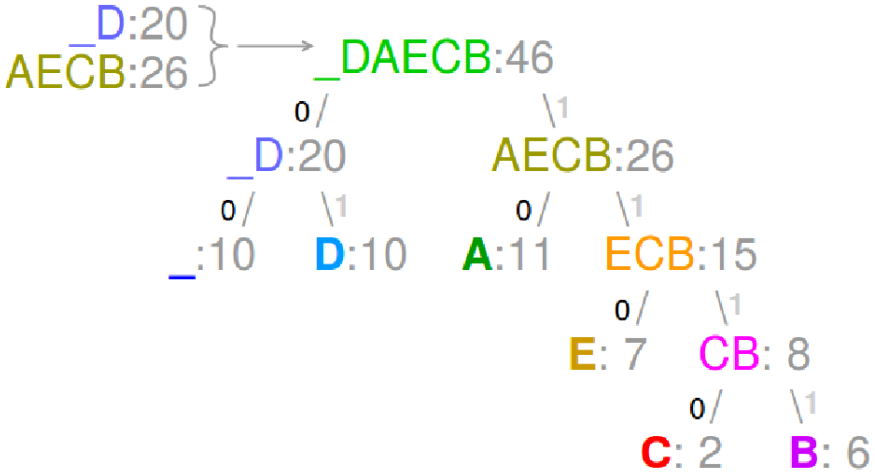
4.



5.



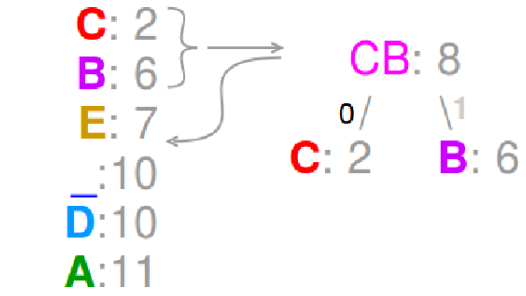
6.



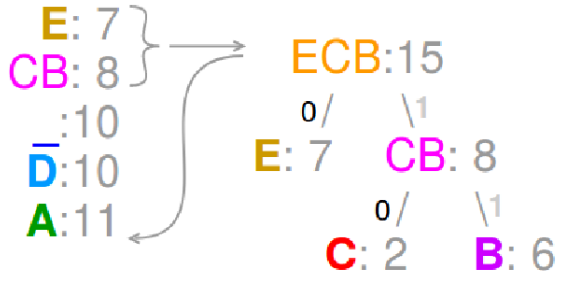
Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

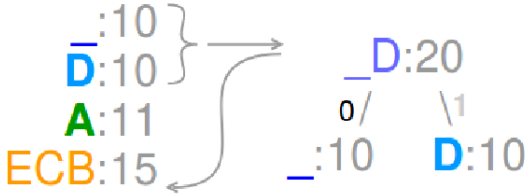
2.



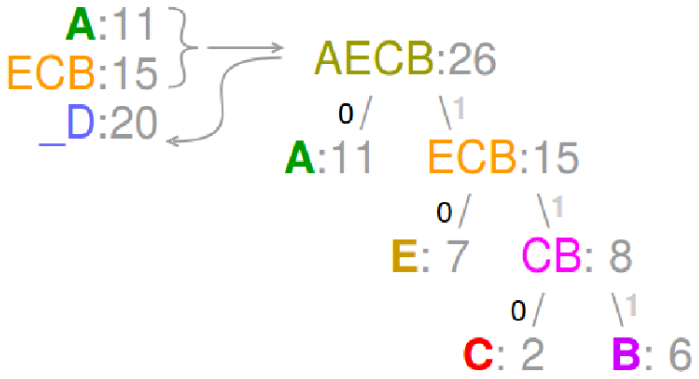
3.



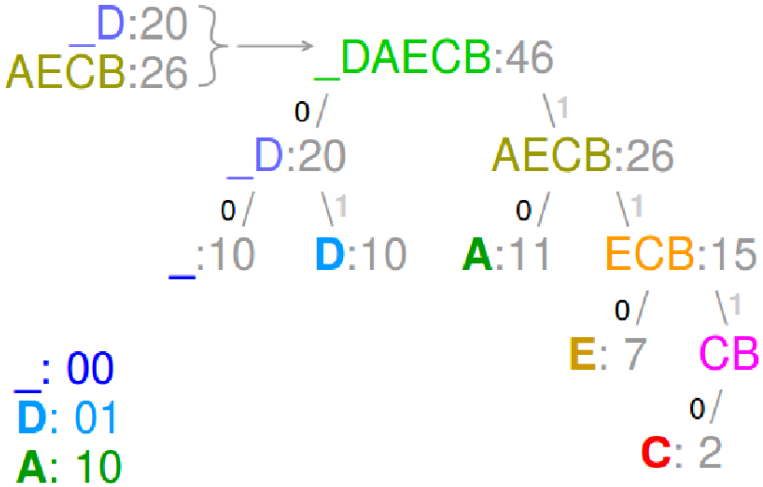
4.



5.



6.



7.
_: 00
D: 01
A: 10
E: 110
C: 1110
B: 1111

Huffmann coding by example – from [wikipedia]

1. "A_DEAD_DAD_CEDED_A_BAD_BABE_A_BEADED_ABACA_BED"

2.
```
C: 2
B: 6
E: 7
_:10
D:10
A:11
```
→
```
CB: 8
0/   \1
C: 2   B: 6
```

3.
```
E: 7
CB: 8
_:10
D:10
A:11
```
→
```
ECB:15
0/   \1
E: 7   CB: 8
       0/   \1
       C: 2   B: 6
```

4.
```
_:10
D:10
A:11
ECB:15
```
→
```
_D:20
0/   \1
_:10   D:10
```

5.
```
A:11
ECB:15
_D:20
```
→
```
AECB:26
0/   \1
A:11   ECB:15
       0/   \1
       E: 7   CB: 8
              0/   \1
              C: 2   B: 6
```

6.
```
_D:20
AECB:26
```
→
```
_DAECB:46
0/                \1
_D:20             AECB:26
0/   \1           0/   \1
_:10   D:10       A:11   ECB:15
                         0/   \1
                         E: 7   CB: 8
                                0/   \1
                                C: 2   B: 6
```

7.
```
_: 00
D: 01
A: 10
E: 110
C: 1110
B: 1111
```

8. "1000011101001000110010011101100111001001000111110010011111011111100010001111110100111001001011111011101000111111001"

Huffman encoding summary

Construction

1. Compute frequencies of each unique character in the given text/data.
2. Iteratively merge two nodes with two lowest frequencies to form a new internal node.
   - this new internal node's (i.e. subset of characters') frequency is simply the sum of frequencies of the two merged nodes.
   - By convention, of the two nodes being merged, the node with the lowest frequency becomes the left branch of the new internal node and is assigned the bit symbol `0`...
   - ... and the other becomes the right branch, and is assigned the bit symbol `1`.
   - When the frequencies are same, the choice is arbitrary.
3. When all nodes are merged into a rooted binary tree, stop.
4. Each character's codeword is then the sequence of `0`'s and `1`'s generated from `root`-to-`leaf` traversal on that binary tree.

Decoding

Decoding Huffmann encoded bitstream back into text is simple. Why?
(Reason from the encoded string in the example in the previous slide.)

Huffman coding: Optimality

- **Symbols:** n symbols $\{\psi_1, \psi_2, \ldots, \psi_n\}$.
- **Frequencies:** $f_1 \leq f_2 \leq \ldots \leq f_n$.
- **Length of codeword:** The i-th symbol is assigned a binary (tree) code of length l_i.
- **Expected length of the code:** $L(n) = \sum_{i=1}^{n} f_i l_i$.
- **Objective:** Find the code (i.e., set of codewords for symbols) that minimizes $L(n)$.

Greedy choice is optimal

- Merge ψ_1 and ψ_2 (with frequencies f_1 and f_2) into $\psi_{\{1,2\}}$ with frequency $f_{\{1,2\}} = f_1 + f_2$. Why?
  - Merging any other higher-frequency symbol ψ_i $(i \neq 1, 2)$ with either ψ_1 or ψ_2 will increase $L(n)$.
- Therefore, the expected length of code with n symbols satisfies the relationship:

$$L(n) = L(n-1) + f_1 + f_2$$

- Minimizing $L(n)$ reduces to minimizing $L(n-1)$, since $f_1 + f_2$ is constant.
- Hence, **greedy** merging into $\psi_{\{1,2\}}$ yields a smaller subproblem with $n-1$ symbols to optimize.
- Repeat greedy merging with these $n-1$ symbols (subproblem) – this is the Huffman's algorithm.
- Hence, Huffman coding is optimal for n symbols.

What about prefix-free codes for **`integers`**?

- Assume we have a data stream of integers coming from the set of natural numbers.
- Huffman coding is not effective for such data...
- ...especially when integers are spread over large ranges, and each integer in the data stream is nearly unique.
- This motivates designing variable-length, prefix-free codes for integers.

Definition: Minimal binary code

The minimal binary codeword of a number is the binary representation of that number such that the most significant bit is always **1** (that is, there are no 0's padded on the most-significant side).

Example

$$(561)_{dec} = \ldots \overset{15}{0}\ \overset{14}{0}\ \overset{13}{0}\ \overset{12}{0}\ \overset{11}{0}\ \overset{10}{0}\ \overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}\ .$$

The minimal binary code of $(561)_{dec} = \overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}\ .$

Elias Omega code for positive integers

Elias Omega encoding method assigns prefix-free binary 'codewords' for any positive integer $1, 2, \ldots, \infty \in \mathbb{Z}_{>0}$.

Overarching strategy

The codeword for any positive integer $N > 0$ is of the form:

.....	.....	binary(L_3)	binary(L_2)	binary(L_1)	binary(N)

where binary(any positive integer) gives the **minimal binary codeword** of that integer*, and

- L_1=length(binary(N)) -1
- L_2=length(binary(L_1)) -1
- L_3=length(binary(L_2)) -1
- ...
- ... and so on until the final encoded length is only one bit long.

We will call the binary codes for L_1, L_2, L_3 $\cdots$ as **length components** and that of N as the **code component**.

---

*Important! The most significant bit of each **length component** is flipped from $1 \to 0$ to make the Elias codeword uniquely decodable – see slide 23

Elias (Omega) code for positive integers..continued

Decoding can be problematic using just minimal binary codes

Encoding $N, L_1, L_2, \ldots$ integers using directly their minimal binary codes poses a problem.

- During decoding, we cannot differentiate between the **length components** and the actual **code component** of N.

Elias (Omega) code for positive integers..continued

Decoding can be problematic using just minimal binary codes

Encoding $N, L_1, L_2, \ldots$ integers using directly their minimal binary codes poses a problem.

- During decoding, we cannot differentiate between the **length components** and the actual **code component** of N.

Example $N = 561$ (listen to the lecture)

Decoding
`11110011000110001`

Elias (Omega) code for positive integers..continued

Decoding can be problematic using just minimal binary codes

Encoding $N, L_1, L_2, \ldots$ integers using directly their minimal binary codes poses a problem.

- During decoding, we cannot differentiate between the **length components** and the actual **code component** of N.

Example $N = 561$ (listen to the lecture)

Decoding
`11110011000110001`

....requires its interpretation as the following length and code components

$$\underbrace{1}_{L_3}\ \underbrace{11}_{L_2}\ \underbrace{1001}_{L_1}\ \underbrace{1000110001}_{N}.$$

In general, it is impossible to delineate the string of concatenated bits into its length and code components.

Elias (Omega) code for positive integers..continued

Decoding can be problematic using just minimal binary codes

Encoding $N, L_1, L_2, \ldots$ integers using directly their minimal binary codes poses a problem.

- During decoding, we cannot differentiate between the **length components** and the actual **code component** of N.

Example $N = 561$ (listen to the lecture)

Decoding
`11110011000110001`

....requires its interpretation as the following length and code components

$$\underbrace{1}_{L_3}\ \underbrace{11}_{L_2}\ \underbrace{1001}_{L_1}\ \underbrace{1000110001}_{N}.$$

In general, it is impossible to delineate the string of concatenated bits into its length and code components.

Elias (omega) encoding of integers addresses this problem by changing the most-significant 1 in the minimal binary code of each **length component** to 0.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$
- The minimal binary code for L_1 is therefore $\overset{3}{1}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$
- The minimal binary code for L_1 is therefore $\overset{3}{1}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.
- IMPORTANT RULE: When coding any **length component**, change the leading 1 of its minimal binary code to 0.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$
- The minimal binary code for L_1 is therefore $\overset{3}{1}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.
- IMPORTANT RULE: When coding any **length component**, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_1 is now $\overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$
- The minimal binary code for L_1 is therefore $\overset{3}{1}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.
- IMPORTANT RULE: When coding any **length component**, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_1 is now $\overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- Let $N = (561)_{dec}$.
- The minimal binary code of N is $\overset{9}{1}\ \overset{8}{0}\ \overset{7}{0}\ \overset{6}{0}\ \overset{5}{1}\ \overset{4}{1}\ \overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$
- The **length** of minimal binary code of N is $(10)_{dec}$.
- Therefore, the **length component** to encode is $L_1 = (10 - 1 = 9)_{dec}$
- The minimal binary code for L_1 is therefore $\overset{3}{1}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.
- IMPORTANT RULE: When coding any **length component**, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_1 is now $\overset{3}{0}\ \overset{2}{0}\ \overset{1}{0}\ \overset{0}{1}$.

Encoded string so far

$$\underbrace{0001}_{L_1}\ \underbrace{1000110001}_{N}$$

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathbb{0}}\ \overset{2}{\mathbb{0}}\ \overset{1}{\mathbb{0}}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathbb{0}}\ \overset{2}{\mathbb{0}}\ \overset{1}{\mathbb{0}}\ \overset{0}{1}$.
- The length of (modified) L_1 is 4.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathtt{0}}\ \overset{2}{\mathtt{0}}\ \overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathtt{0}}\ \overset{2}{\mathtt{0}}\ \overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.
- The minimal binary code for L_2 is therefore $\overset{1}{\mathtt{1}}\ \overset{0}{\mathtt{1}}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathbf{0}}\ \overset{2}{\mathbf{0}}\ \overset{1}{\mathbf{0}}\ \overset{0}{1}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.
- The minimal binary code for L_2 is therefore $\overset{1}{1}\ \overset{0}{1}$.
- Again, since this is a **length** component, as per the rule stated earlier, change the leading 1 of its minimal binary code to 0.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathbf{0}}\ \overset{2}{\mathbf{0}}\ \overset{1}{\mathbf{0}}\ \overset{0}{1}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.
- The minimal binary code for L_2 is therefore $\overset{1}{1}\ \overset{0}{1}$.
- Again, since this is a **length** component, as per the rule stated earlier, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_2 is now $\overset{1}{\mathbf{0}}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathbf{0}}\ \overset{2}{\mathbf{0}}\ \overset{1}{\mathbf{0}}\ \overset{0}{1}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.
- The minimal binary code for L_2 is therefore $\overset{1}{1}\ \overset{0}{1}$.
- Again, since this is a **length** component, as per the rule stated earlier, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_2 is now $\overset{1}{\mathbf{0}}\ \overset{0}{1}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_1 is now $\overset{3}{\mathtt{0}}\ \overset{2}{\mathtt{0}}\ \overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_1 is 4.
- The next **length component** to encode is therefore $L_2 = (4 - 1 = 3)_{dec}$.
- The minimal binary code for L_2 is therefore $\overset{1}{\mathtt{1}}\ \overset{0}{\mathtt{1}}$.
- Again, since this is a **length** component, as per the rule stated earlier, change the leading 1 of its minimal binary code to 0.
- That is, the code to encode (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.

Encoded string so far

$$\underbrace{\mathtt{01}}_{L_2}\ \underbrace{\mathtt{0001}}_{L_1}\ \underbrace{\mathtt{1000110001}}_{N}$$

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_2 is 2.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\cancel{0}}\ \overset{0}{\cancel{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{0}{\cancel{1}}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{0}{\mathtt{1}}$.
- Since this is a **length component**, as per the rule stated above, change the leading 1 of the minimal binary code to 0.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathbb{0}}\ \overset{\mathbb{0}}{1}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{\mathbb{0}}{1}$.
- Since this is a **length component**, as per the rule stated above, change the leading 1 of the minimal binary code to 0.
- That is, the code to encode (modified) L_3 is now $\overset{1}{\mathbb{0}}$.

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathbf{0}}\ \overset{0}{\mathbf{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{0}{\mathbf{1}}$.
- Since this is a **length component**, as per the rule stated above, change the leading 1 of the minimal binary code to 0.
- That is, the code to encode (modified) L_3 is now $\overset{1}{\mathbf{0}}$.
- Since the length of (modified) L_3 has reached 1, STOP encoding!

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathbf{0}}\ \overset{0}{\mathbf{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{0}{\mathbf{1}}$.
- Since this is a **length component**, as per the rule stated above, change the leading 1 of the minimal binary code to 0.
- That is, the code to encode (modified) L_3 is now $\overset{1}{\mathbf{0}}$.
- Since the length of (modified) L_3 has reached 1, STOP encoding!

Elias (omega) code for positive integers – Encoding via an example

- From previous slide, (modified) L_2 is now $\overset{1}{\mathtt{0}}\ \overset{0}{\mathtt{1}}$.
- The length of (modified) L_2 is 2.
- The next **length component** to encode is therefore $L_3 = (2 - 1 = 1)_{dec}$.
- The minimal binary code for L_3 is therefore $\overset{0}{\mathtt{1}}$.
- Since this is a **length component**, as per the rule stated above, change the leading 1 of the minimal binary code to 0.
- That is, the code to encode (modified) L_3 is now $\overset{1}{\mathtt{0}}$.
- Since the length of (modified) L_3 has reached 1, STOP encoding!

Final encoded bit string

$$\underbrace{\mathtt{0}}_{L_3}\ \underbrace{\mathtt{01}}_{L_2}\ \underbrace{\mathtt{0001}}_{L_1}\ \underbrace{\mathtt{1000110001}}_{N}$$

Elias (omega) codewords for the first few integers

integer N	**Components (incl.** N**)**	**codeword**
1	1	1
2	1,2	0 10
3	1,3	0 11
4	1,2,4	0 00 100
5	1,2,5	0 00 101
6	1,2,6	0 00 110
7	1,2,7	0 00 111
8	1,3,8	0 01 1000
9	1,3,9	0 01 1001
10	1,3,10	0 01 1010
...		
15	1,3,15	0 01 1111
16	1,2,4,16	0 00 000 10000
...		

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]

Example worked out during the lecture.

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]
2. Initialize: readlen $= (1)_{dec}$, component = <EMPTY>, $pos = 1$

Example worked out during the lecture.

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]
2. Initialize: readlen $= (1)_{dec}$, component = <EMPTY>, $pos = 1$
3. component = codeword$[pos \ldots pos + \texttt{readlen} - 1]$.

Example worked out during the lecture.

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]
2. Initialize: readlen $= (1)_{dec}$, component = <EMPTY>, $pos = 1$
3. component = codeword$[pos \ldots pos + \texttt{readlen} - 1]$.
4. If the **most-significant** bit of component is 1, then $N = (\texttt{component})_{dec}$. STOP.

Example worked out during the lecture.

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]
2. Initialize: $\texttt{readlen} = (1)_{dec}$, $\texttt{component} = \texttt{<EMPTY>}$, $pos = 1$
3. $\texttt{component} = \texttt{codeword}[pos \ldots pos + \texttt{readlen} - 1]$.
4. If the **most-significant** bit of component is 1, then $N = (\texttt{component})_{dec}$. STOP.
5. Else, if the **most-significant** bit of component is 0, then flip $0 \rightarrow 1$ and reset $pos = pos + \texttt{readlen}$, $\texttt{readlen} = (\texttt{component})_{dec} + 1$.

Example worked out during the lecture.

Elias code for positive integers – Decoding example

- When decoding a variable-length encoded integer, we do not know *a priori*:
  - how many **length components**, $L_k, L_{k-1}, \ldots L_1$, the codeword has...
  - ...and what N is.
- We also know is that the very last **length component**, L_k, in the codeword has length $= (1)_{dec}$, and it is encoded with 0 bit. (See slide 26.).

Decoding method: Input variable-length codeword of N

1. Input: codeword[1...]
2. Initialize: $\texttt{readlen} = (1)_{dec}$, $\texttt{component} =$ <EMPTY>, $pos = 1$
3. $\texttt{component} = \texttt{codeword}[pos \ldots pos + \texttt{readlen} - 1]$.
4. If the **most-significant** bit of component is 1, then $N = (\texttt{component})_{dec}$. STOP.
5. Else, if the **most-significant** bit of component is 0, then flip $0 \rightarrow 1$ and reset $pos = pos + \texttt{readlen}$, $\texttt{readlen} = (\texttt{component})_{dec} + 1$.
6. Repeat from step (3) (until N is decoded when step (4) is true).

Example worked out during the lecture.

What about codewords for non-negative integers, $\mathbb{Z}_{\geq 0}$?

- The set of positive integers ($\mathbb{Z}_{>0}$) is **isomorphic** to the set of non-negative integers ($\mathbb{Z}_{\geq 0}$).
- They are both **countable** sets that share a one-to-one correspondence (bijection).

Elias omega codewords for non-negative integers

For any integer $z \in \mathbb{Z}_{\geq 0}$, assign it the Elias Omega codeword for $z + 1$ in positive range. That is:

- Codeword for $z = 0$ in $\mathbb{Z}_{\geq 0}$ is same as the codeword for $z = 1$ in $\mathbb{Z}_{>0}$.
- Codeword for $z = 1$ in $\mathbb{Z}_{\geq 0}$ is same as the codeword for $z = 2$ in $\mathbb{Z}_{>0}$.
- Codeword for $z = 2$ in $\mathbb{Z}_{\geq 0}$ is same as the codeword for $z = 3$ in $\mathbb{Z}_{>0}$.
- Codeword for $z = 3$ in $\mathbb{Z}_{\geq 0}$ is same as the codeword for $z = 4$ in $\mathbb{Z}_{>0}$.
- $\vdots$
- ..and so on.

Lempel Ziv algorithms for lossless text compression

Lempel-Ziv (LZ77) algorithm

LZ77

- LZ77 is a **`sliding window`** based algorithm.
- Since original publication, it inspired many variants (eg. LZSS)
- Used in many applications: `gzip`, **`PKZIP`** etc.

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **`longest`** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **`longest`** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)
  - This yields three pieces of information:

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **`longest`** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)
  - This yields three pieces of information:
    1. the offset (i.e., distance of the match from the current char/substring being encoded).

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **longest** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)
  - This yields three pieces of information:
    1. the offset (i.e., distance of the match from the current char/substring being encoded).
    2. the length of the match.

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **`longest`** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)
  - This yields three pieces of information:
    1. the offset (i.e., distance of the match from the current char/substring being encoded).
    2. the length of the match.
    3. the next character `char` in the lookahead buffer, after the matched char/substring.

LZ77 basic strategy

- The LZ77 encoding involves examining the input text through a sliding window.
- The window consists of 2 consecutive parts:
  1. search window (also called the 'dictionary')
  2. lookahead buffer (sometimes simply called the 'buffer')
- To encode any char/substring in the lookahead buffer:
  - find the **`longest`** matched substring starting in the dictionary (i.e., search window), and ending before the end of the lookahead buffer.

    (The longest possible length is the size of lookahed buffer.)
  - This yields three pieces of information:
    1. the offset (i.e., distance of the match from the current char/substring being encoded).
    2. the length of the match.
    3. the next character `char` in the lookahead buffer, after the matched char/substring.
- Using this search, the char/substring at the current position is encoded as a **triple** ⟨offset,length,`char`⟩

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

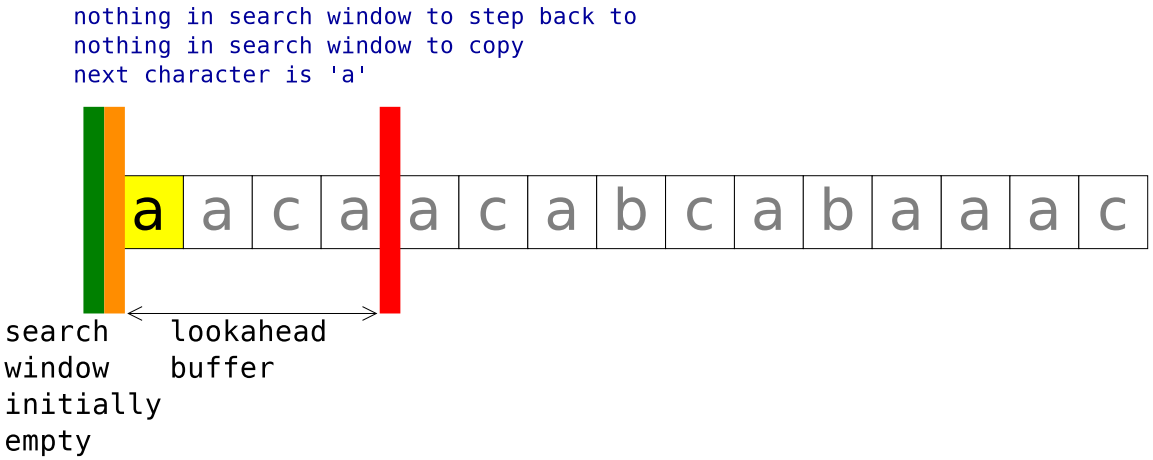
iter-0

a	a	c	a	a	c	a	b	c	a	b	a	a	a	c

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

iter-1

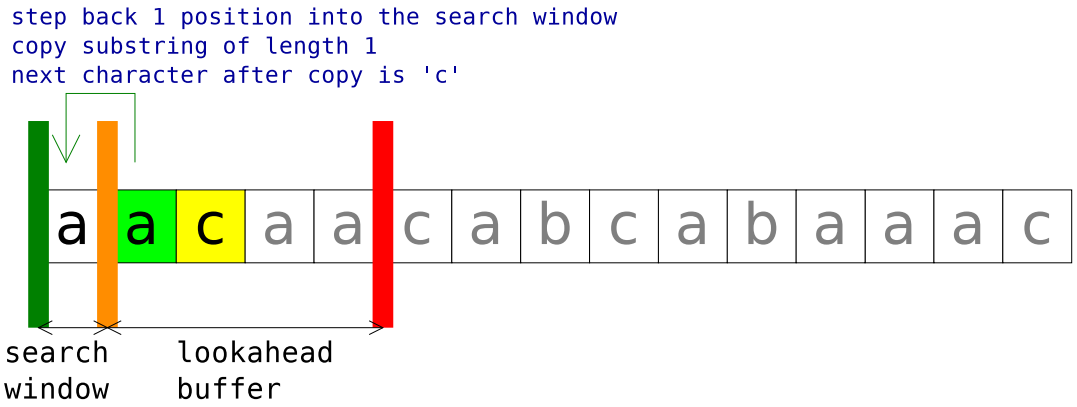


offset	len	next-char
0	0	'a'

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

iter-2

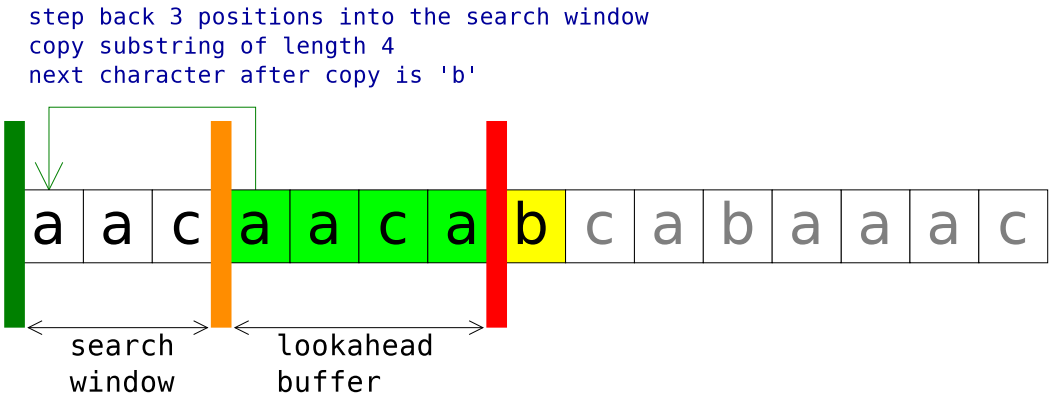


offset	len	next-char
[1,	1,	'c']

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

iter-3

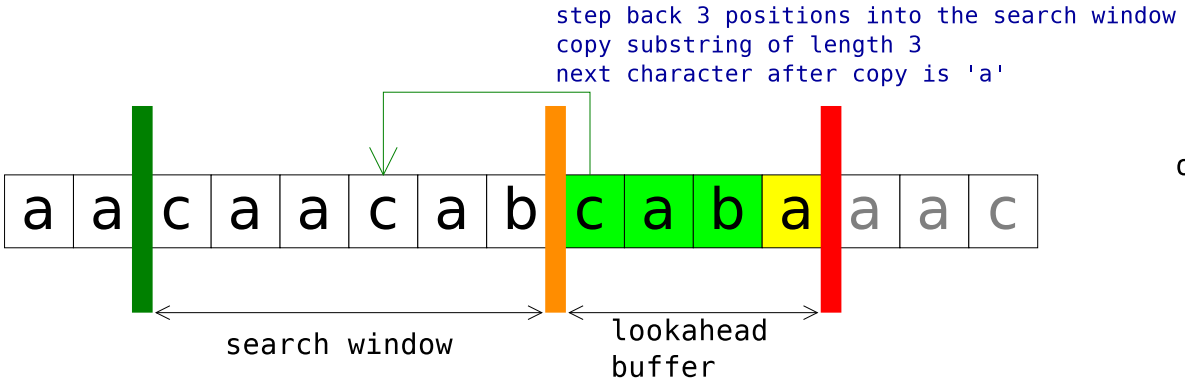


offset	len	next-char
[3,	4,	'b']

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

iter-4



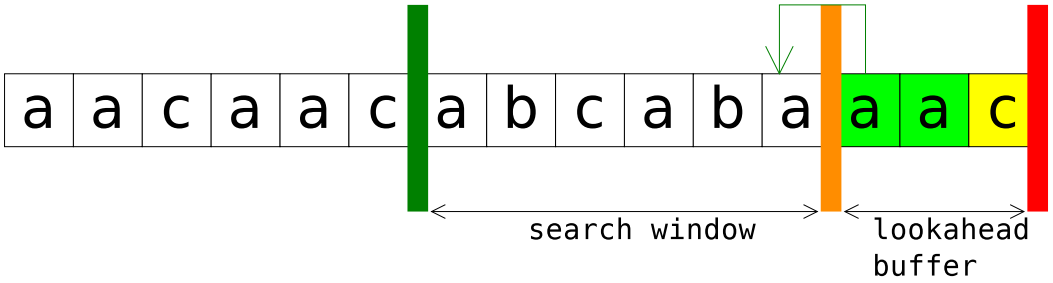
offset	len	next-char
3	3	'a'

LZ77 encoding example – explained during lecture

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

step back 1 position into the search window
copy substring of length 2
next character after copy is 'c'

iter-5



offset	len	next-char
1	2	'c'

LZ77 tuples are represented as a binary stream of bits on a computer

We saw in the previous example, the LZ77 encoding of the string **aacaacabcabaaac** resulted in 5 tuples, each containing two integers (for offset and length) and one character (for encoding the next character after a copy).

- `[0,0, 'a']`,
- `[1,1, 'c']`,
- `[3,4, 'b']`,
- `[3,3, 'a']`,
- `[1,2, 'c']`.

On a computer, these are encoded using variable length integer codes (e.g. Elias Omega code for non-negative integers) and variable length character codes (e.g. Huffman code we learnt in this lecture).

LZ77 tuples are represented as a binary stream of bits on a computer...cont'd

Using Elias omega code for non-negative integers and the following Huffman codewords, 'a' $= 1$, 'b' $= 00$, 'c' $= 01$, we get the following binary representation for each tuple:

- [0,0, 'a'] $\rightarrow$ **111**
- [1,1, 'c'] $\rightarrow$ **01001001**
- [3,4, 'b'] $\rightarrow$ **00010000010100**
- [3,3, 'a'] $\rightarrow$ **0001000001001**
- [1,2, 'c'] $\rightarrow$ **01001101**

Thus, these LZ77 binary encodings over all tuples concatenated defines the following stream of bits packed into Byte-sized chunks on your computer:

11101001	00100010	00001010	00001000	00100101	001101--
Byte-1	Byte-2	Byte-3	Byte-4	Byte-5	Byte-6

An additional header that informs the length of the text/string and Huffman codewords is prepended (also in raw binary) to the above bytes to make the decoding lossless.

Thought bubble

If you ever wondered why you saw gibberish on your screen when you opened some zip file in a PLAIN TEXT editor, it is because you are seeing a packed representation being rendered by your plain text editor as 8-bit ASCII characters.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
   offset  len next-char
[0, 0,'a']
```

Decoded string thus far

?

1

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[0, 0,'a']
```

Decoded string thus far

?

1

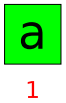
offset and length both 0 implies nothing to copy from previously decoded string

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[0, 0,'a']
```

Decoded string thus far



offset and length both 0 implies nothing to copy from previously decoded string

next-char 'a' decodes the first character of the string

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 1,'c']
```

Decoded string thus far

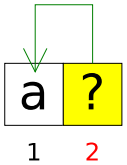
a	?
1	2

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 1,'c']
```

Decoded string thus far



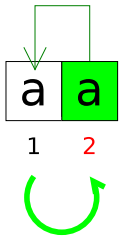
Offset = 1 implies step back 1 position

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 1,'c']
```

Decoded string thus far



Offset = 1 implies step back 1 position

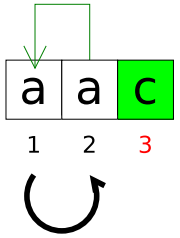
Len = 1 implies copy a substring, char-by-char, of length 1.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 1,'c']
```

Decoded string thus far



Offset = 1 implies step back 1 position

Len = 1 implies copy a substring, char-by-char, of length 1.

next-char = 'c' implies decode and append 'c'

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far

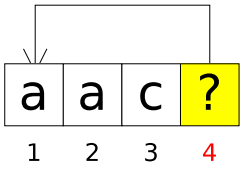
a	a	c	?
1	2	3	4

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
  [3, 4,'b']
```

Decoded string thus far



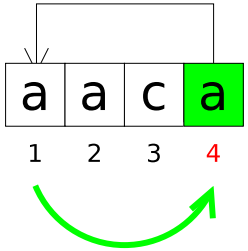
Offset = 3 implies step back 3 positions

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

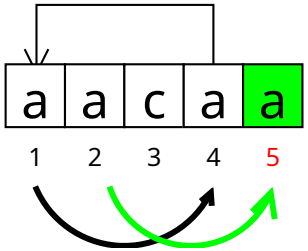
Length= 4 implies copy substring, char-by-char, of length 4.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

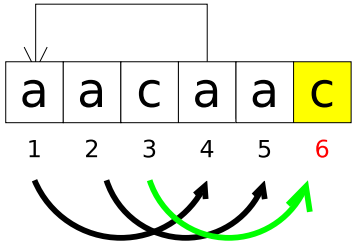
Length= 4 implies copy substring, char-by-char, of length 4.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

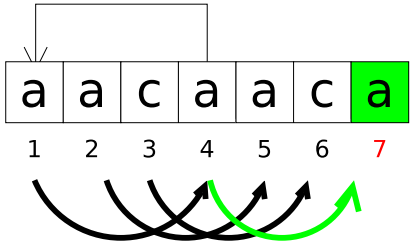
Length= 4 implies copy substring, char-by-char, of length 4.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

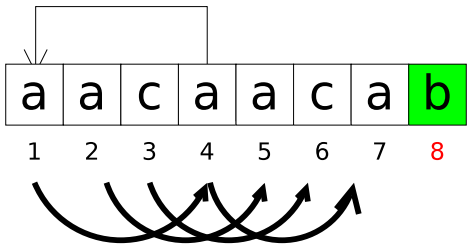
Length= 4 implies copy substring, char-by-char, of length 4.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 4,'b']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

Length= 4 implies copy substring, char-by-char, of length 4.

Next-char = 'b' implies decode and append 'b'

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far

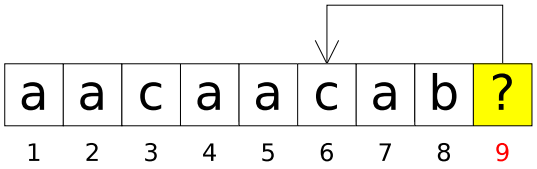
a	a	c	a	a	c	a	b	?
1	2	3	4	5	6	7	8	9

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far



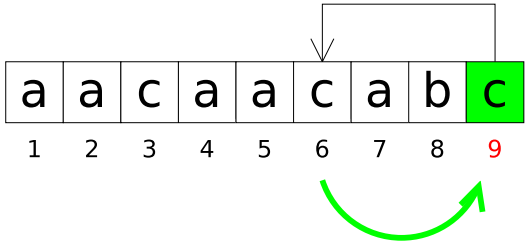
Offset = 3 implies step back 3 positions

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

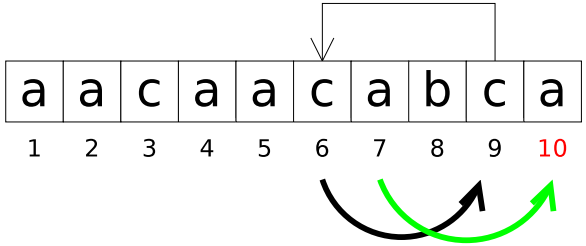
Length= 3 implies copy substring, char-by-char, of length 3.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

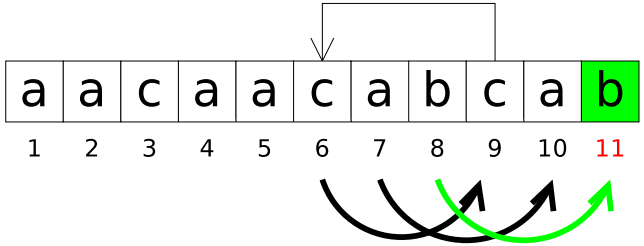
Length= 3 implies copy substring, char-by-char, of length 3.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

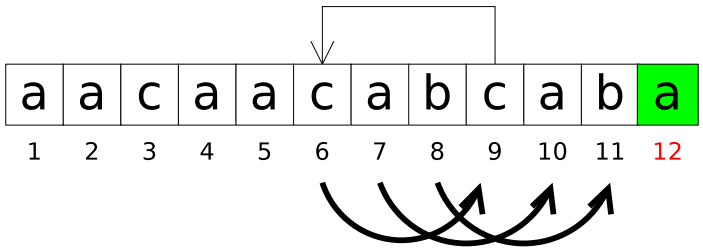
Length= 3 implies copy substring, char-by-char, of length 3.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[3, 3,'a']
```

Decoded string thus far



Offset = 3 implies step back 3 positions

Length= 3 implies copy substring, char-by-char, of length 3.

Next-char = 'a' implies decode and append 'a'

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 2,'c']
```

Decoded string thus far

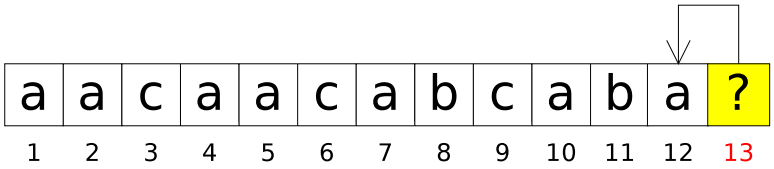
a	a	c	a	a	c	a	b	c	a	b	a	?
1	2	3	4	5	6	7	8	9	10	11	12	13

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 2,'c']
```

Decoded string thus far



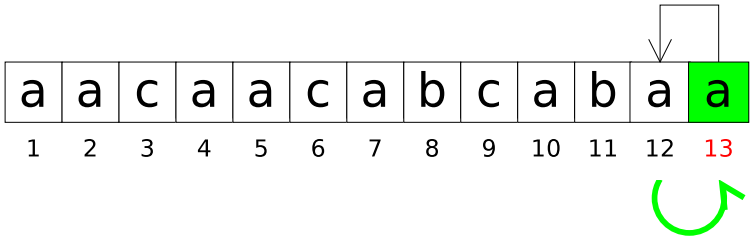
Offset = 1 implies step back 1 positions

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 2,'c']
```

Decoded string thus far



Offset = 1 implies step back 1 positions

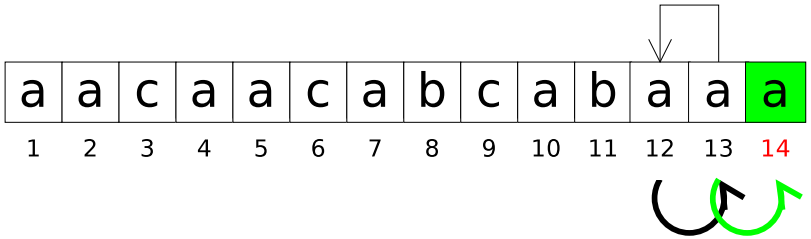
Length = 2 implies copy substring, char-by-char, of length 2.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 2,'c']
```

Decoded string thus far



Offset = 1 implies step back 1 positions

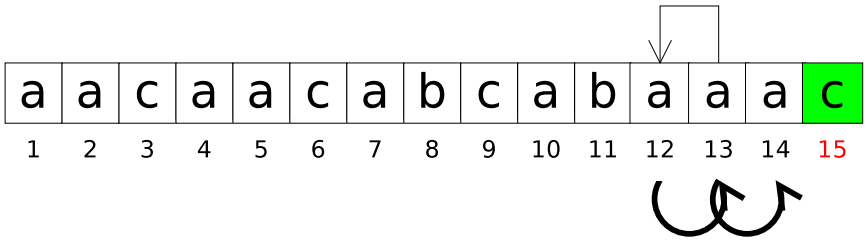
Length = 2 implies copy substring, char-by-char, of length 2.

LZ77 decoding (same example used for encoding) – explained during lecture

LZ77 tuple

```
offset  len next-char
[1, 2,'c']
```

Decoded string thus far



Offset = 1 implies step back 1 positions

Length = 2 implies copy substring, char-by-char, of length 2.

Next-char = 'c' implies decode and append 'c'

LZ77 decoding (same example used for encoding) – explained during lecture

Final decoded string

a	a	c	a	a	c	a	b	c	a	b	a	a	a	c
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

LZSS (Lempel-Ziv-Storer-Szymanski) empirically-inspired variation of LZ77 algorithm

LZSS variant of LZ77 is a simple attempt to reduce the number of bits needed to compress using the original LZ77. Instead of always using $\langle \text{offset}, \text{length}, \text{char} \rangle$-tuple based encoding, LZSS instead encodes each of its tuples in one of two formats:

1. `Format` 1 for short copies: When the **`length`** of the matched substring in the dictionary $<\delta$, use $\langle bit\ 1$, `char` $\rangle$-tuple format.
2. `Format` 0 for longer copies: When the **`length`** of the matched substring in the dictionary $\geq\delta$, use $\langle bit\ 0$, offset, length $\rangle$-tuple format.

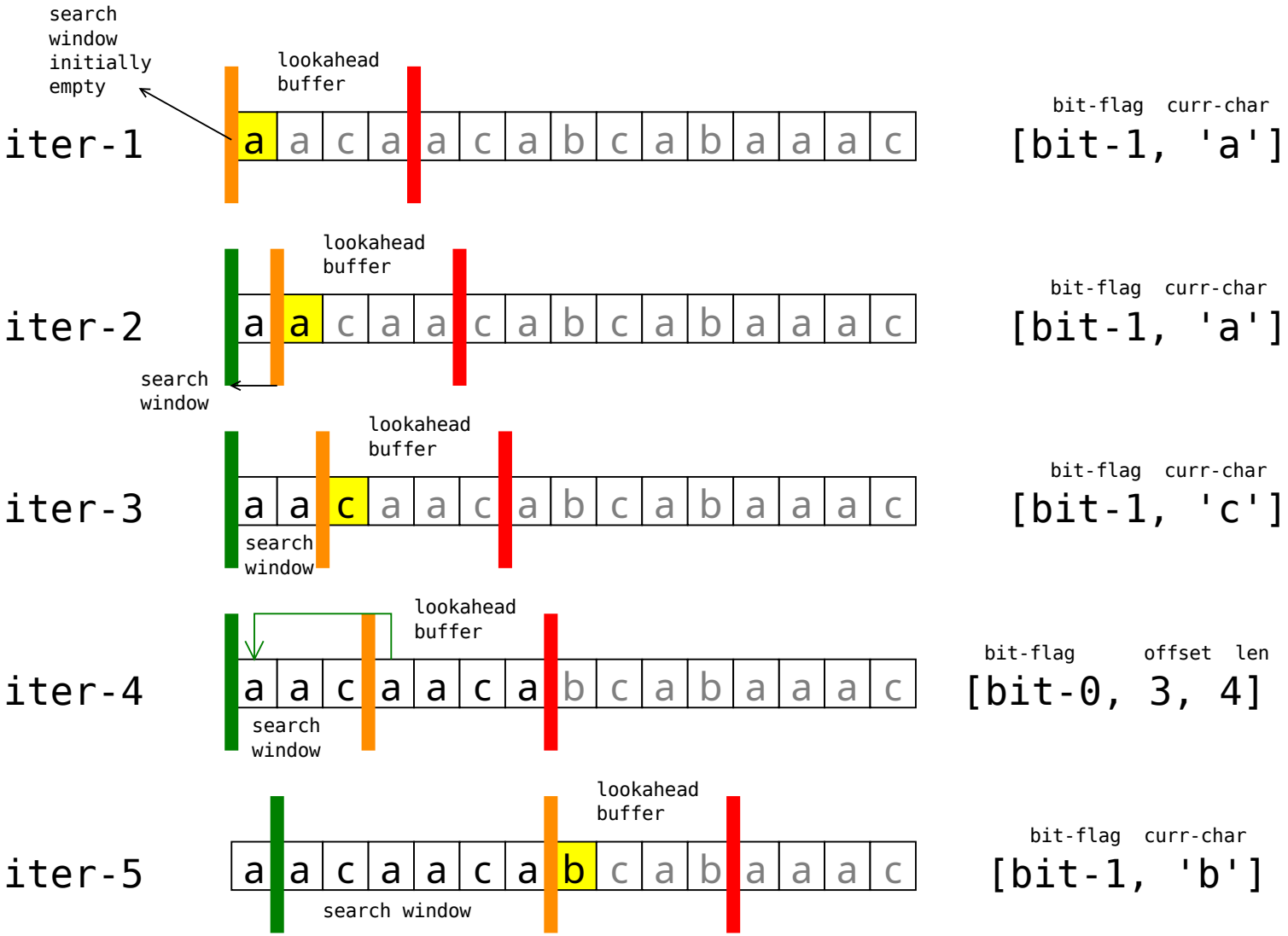
The default value of δ used by LZSS is 3 for plain English text (based on its empirical properties).

Motivation

For short length copies, LS77 does not gain much over the raw (fixed-width) encoding of the characters, as it requires stating the offset (which can be a large integer) can take tens of bits in practice. The switch between encoding short-length copies and those that are not is an attempt to save bits – this is really 'bit-picking'.

LZSS encoding example

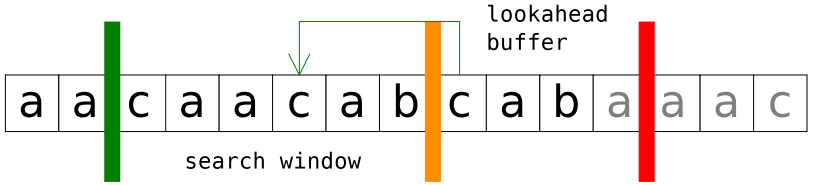
```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```



LZSS encoding example

```
Encoding parameters:
Max search window size = 6
Max lookahead buffer size = 4
```

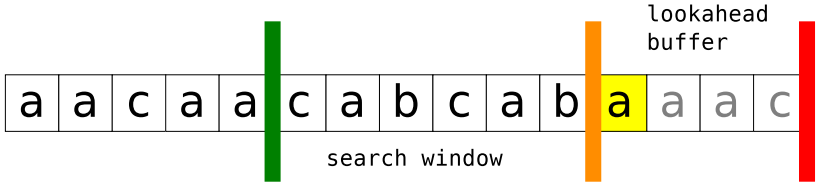
iter-6



bit-flag offset len

[bit-0, 3, 3]

iter-7



bit-flag curr-char

[bit-1, 'a']

...and so on

Self-study

During self-study, explore what optimizations can be applied towards implementing an encoder and decoder for LZ77 and LZSS variant, using the variable-length codes we have studied here.

Ask yourself during this if the algorithms we have learnt in previous weeks could help optimize your implementation.

Next week

Semi-Numerical/Number-theoretic Algorithms

```
-=o0o=-
  END
-=o0o=-
```

Appendix: Outcomes and Probability of outcomes – non-examinable

Random variable and outcomes

- A **random variable** x is a variable...
- ...that can take a set of *possible* values, $o_1, o_2, \cdots, o_n$.
- Each such possible value is an **outcomes** of some (random) event/phenomenon. Examples:
  - Throw of dice
  - Coin tosses
  - Occurrence of a character in text.
  - etc.
- Associated with each outcome $x = o_i$, there is a **probability** of that outcome, denoted by $\Pr(x = o_i)$.

Appendix: How do we measure information content of an outcome $x = o_i$? – non-examinable

Among the outstanding contributions of Claude E. Shannon is his work on Mathematical theory of communication

Shannon's **information content** of an outcome

The measure of the information content of an **outcome** $x = o_i$ of a random variable x is given by:

$$I(o_i) = -\log(\Pr(x = o_i))$$

If the base of the log is 2, $I(.)$ is measured in **bits**.

Shannon's **entropy** over all outcomes

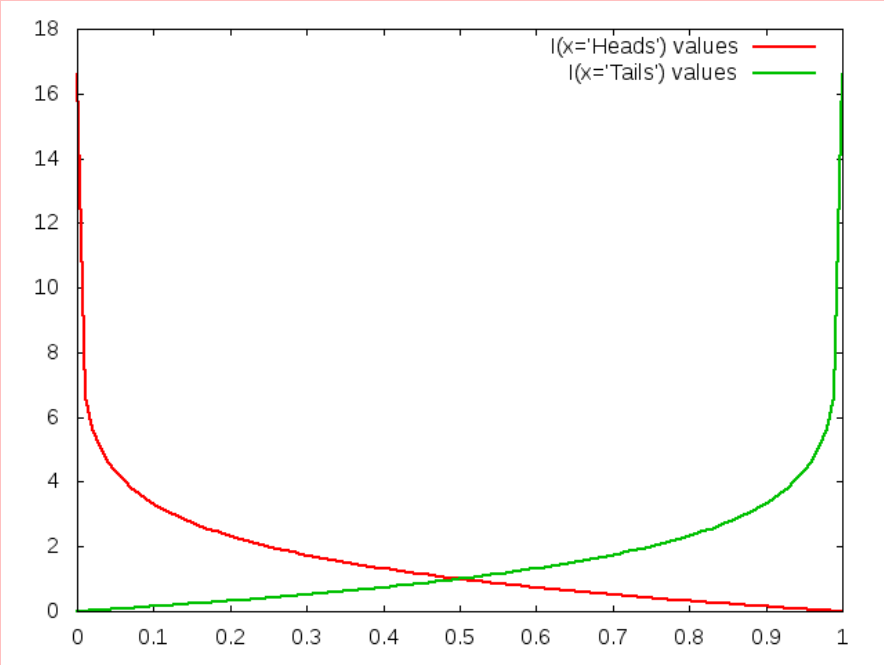
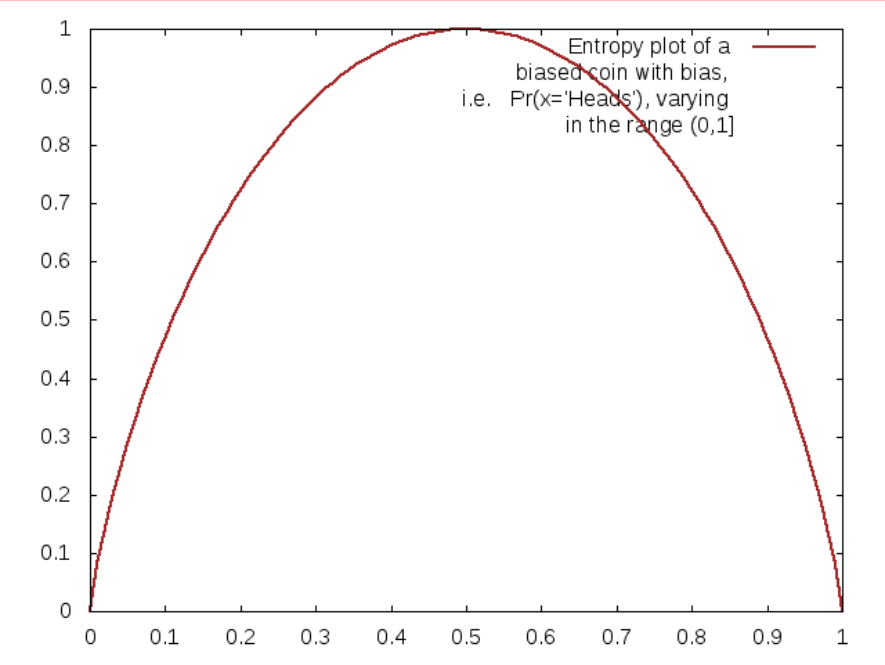
Shannon's **entropy** gives the measure of the **average** information content across all outcomes of the random variable x:

$$H(x) = \sum_{i=1}^{n} \Pr(x = o_i) I(o_i)$$

Example: Coin with a bias – non-examinable

- Assume we have coin with a bias p.
- That is, $\Pr(x = \texttt{Heads}) = p$); $\Pr(x = \texttt{Tails}) = 1 - p$).
- Shannon's information content:
  - $I(\texttt{Heads}) = -\log_2(p)$
  - $I(\texttt{Tails}) = -\log_2(1-p)$
- Shannon's Entropy: $H(x) = p \times I(\texttt{Heads}) + (1-p) \times I(\texttt{Tails})$.

Information content plot



Entropy plot